ChatGPT

面向 NeurIPS [1] 2026 的 Function Calling 立项严审与重构报告

方案审查

你这份草案的“可被顶会审稿人当成论文主张来评判”的核心命题，其实只有一个：**把 function calling 的学习对象从“表层 tool-call tokens”上移到“schema-invariant 的 latent decision”，并且让执行反馈主要用于更新（或蒸馏）decision posterior，而不是做 token-level repair**。这个命题落地后，自然带出两条可检验的子主张：其一，latent decision 在等价 schema 扰动下应该稳定；其二，**execution feedback → posterior refinement** 这种更新路径在跨 schema / unseen tool setting 下应该优于 token-level hindsight/self-distillation。你草案里 A/B/C 的取舍（A 方法、B 证据链、C 可选扩展）是清醒的，但“能不能站得住”取决于：你能否把 A 的 novelty 从“把 SDPO/OpenClaw 这些 token-hindsight 换个空间”里硬生生拽出来，并且用 B 的实验把这种区别钉死。

从现有公开工作看，这条线正好踩在三个强对比点上：
- **Latent action / latent policy 的叙事已很成熟**：例如 CoLA 明确把 LLM 的 action space 抽象到 latent，并用 inverse dynamics 从 future tokens 抽取 latent actions，再把 LLM 当 world model 来做 RL/BC。[2]
- **Rich feedback → token-level dense signal 的叙事也已很成熟**：SDPO 把可验证环境提供的文本反馈转成“反馈条件下的 self-teacher”，再把 feedback-informed next-token 分布蒸馏回 policy；它明确报告覆盖 tool use 等环境。[3]
- **“命名/格式/无关工具”导致的鲁棒性问题，已被 function calling 社区当成核心痛点**：Hammer 直接点名“对 naming convention 过拟合导致跨 benchmark 波动”，并用 function masking + 数据增强去缓解；BFCL v4 也直接把 format sensitivity 系统化成评测子项目。[4]

换句话说：**你想解决的问题是真问题**（命名/格式/扰动鲁棒性 + 执行反馈学习），但你想讲的“方法新意”极容易被现有叙事吞掉：CoLA（latent action）+ SDPO/OpenClaw（rich feedback distillation/OPD）+ Hammer（naming overfit 的数据/掩码工程）拼一起，审稿人会天然怀疑这是“空间替换/recipe engineering”。[5]

你草案宣称“相对于 CoLA adaptation / token hindsight 换空间的增量”，在我看来**只有在下面这条被你证明得足够硬时才成立**：
- **增量必须是一个“可形式化的 invariance/equivariance 结构”**：decision 表示对 schema 变换（tool rename / arg rename / doc paraphrase / shape transform）应当近似不变，而 grounding 对 schema 变换应当等变（equivariant）。否则，latent decision 这件事会被理解为“ToolWeaver/Toolscaler 式的工具标识压缩/编码”或“CoLA 风格 latent action 迁移”。[6]
- **execution feedback 的作用点必须被证明是“decision posterior”而不是“token repair in disguise”**：SDPO/OpenClaw 的 token-level distillation 也能吃到很多来自 error message / tool output 的收益；你必须在跨 schema 迁移上证明“同样的反馈，你的更新路径更稳、更可迁移”。[7]

相对相关工作的真实增量与“看起来新但其实不新”的区分

你列的对比系里，最危险的三组“可被审稿人一票否决的归因”如下。

容易被归因为 CoLA adaptation 的部分
CoLA 的关键机制包含：latent action space、inverse dynamics 从未来 token 抽 latent、让 LLM 作为 world model 接收 latent 并训练 policy（BC 或 RL）。[2]

你草案的（s→z，z→y）两层结构 + “posterior matching”的训练形式，在结构上非常像“latent action + inverse model + distill/regularize”。如果你没有给出**function calling 特有的、非平凡的形式化目标**（例如 schema 变换群上的 invariance 约束，或可证明的等价类定义），审稿人会说：这是把 CoLA 的 latent action 搬到 tool calls 上，外加一些 consistency regularization。

容易被归因为 token-level hindsight 换空间的部分
SDPO 把 rich feedback tokenized 后作为 self-teacher context，蒸馏回 policy，目标是缓解 credit assignment 瓶颈。[8]
OpenClaw-RL 也明确把 next-state signals 拆成 evaluative（标量 reward）与 directive（通过 OPD 从 next state 恢复“应该怎么改”），并提供 token-level directional advantage supervision。[9]
你的 “q(z|s,s’) → distill to p(z|s)” 只要没证明 z 真的是 decision，它在审稿人眼里就是：把 SDPO/OPD 从 token logits 蒸馏，换成 latent head 蒸馏——属于“曲线救国但本质相同”。

容易被归因为 Hammer-style/benchmark-style recipe engineering 的部分
Hammer 直接把“命名约定过拟合”当核心问题，并用 function masking + 数据增强（包含 irrelevant function sensitivity）来提升跨 benchmark 泛化。[10]
BFCL v4 的 format sensitivity 把 return format / function doc format / tool call tag / prompt format / prompt style 系统化变成 26 种 prompt 变体，并给出大量模型实证。[11]
如果你的实证主要靠“我们做了 tool renaming / prompt variation / irrelevant tools injection”，但方法本身并没有一个与这些扰动强绑定的、可复现的结构性归因，那么很容易被认为是“加强数据增强/加强prompt/加强约束解码”的工程改良。

定义不清、范围过大、方法堆叠与证据不足的风险点

我认为你草案里有四类“现在就会被 reviewer 卡住”的不清晰点：

1) **“schema-invariant latent decision” 的等价关系没定义**
- tool renaming 属于同构、argument renaming 属于同构，但 argument reshaping 往往改变结构先验（flat→nested / list→dict 等），“语义等价”不再是纯同构，需要你明确“允许哪些结构变换”和“如何判定语义保持”。这部分如果含糊，审稿人会质疑你在做不可控的 task drift。

2) **z 的信息论地位不清：是 decision 还是压缩码**
你自己在风险里写了“latent 可能只是压缩码”，这不是风险陈述，而是审稿人最可能给你的主拒稿理由。要过这一关，你不能只靠“latent consistency score”这类自定义指标；你必须设计**能被反驳/能被证伪**的实验（后面我会给）。否则这块看起来像“我强行让它一致，所以它一致”。

3) **q(z|s,s’) 的构造方式极易滑向“可用反馈把答案直接算出来”的 teacher cheating**
SDPO 的典型质疑是：feedback-informed self-teacher 是否引入了“后门”信息（虽然它动机是 rich feedback）。[12]
你如果不明确限制 posterior 的输入（例如只允许看 execution error 类型而不看完整工具返回；或做条件分层），审稿人会说：你只是让 teacher 看了更多信息，然后蒸馏到 student，当然变好；这不是新的学习对象，只是更多监督。

4) **方法堆叠过多会稀释论文主线**
你草案里同时出现：latent head、grounding decoder、posterior module、schema consistency loss、可选 RL calibration、可选 multi-candidate reranking。对顶会方法论文来说，这很容易被打成“组合拳工程”，除非你能把主线压到一个闭环，并让其余模块在一页内被删掉也不影响主结论。

最可能被审稿人打掉的点

1. **Novelty 归因不成立：会被判定为 CoLA + SDPO/OPD 的“空间替换”**（latent action + feedback distillation 的组合已有成熟叙事）。[13]
2. **“latent decision 不是压缩码”的证据链不够硬**：如果你主要用一致性/聚类这种自证指标，会被认为容易被正则项“做出来”。（这点需要用更强的反事实/置换实验来救。）
3. **B（schema 扰动协议）会被认为是 benchmark engineering 而非方法贡献**：尤其在 BFCL v4 已经系统化 prompt variation 的情况下，你必须解释你新增的扰动集合与 BFCL v4 的关系与差异，否则容易被说“造了个 stress test”。[11]
4. **BFCL 作为训练源会削弱可信度**：不仅因为“benchmark 训练污染”的常规质疑，还因为近期有人专门指出 BFCL-V3 多轮条目存在大量 schema/实现/trajectory 系统性错误并提出修复与 outcome-aware 评估；这意味着你若直接把它当主训练源，会被质疑在学噪声与漏洞。[14]
5. **C（multi-candidate/rerank/diffusion-like）现在做会把论文拉回 inference-time scaling/ verifier 工程**：相关方向已有 Toolken+ 这类 reranking + 校验的工作，且很容易被归到 best-of-n + verifier 的常见套路。[15]

最值得保留的核心命题

1. **“执行反馈应该主要更新 decision-level 表示，而不是只做 token-level 修补”**——这是你 A 线唯一可能形成方法论文主张的地方，且能与 SDPO/OpenClaw 做正面对照。[7]
2. **“schema/format/renaming 鲁棒性应成为一等公民的目标，而不是附属 stress test”**——这与 Hammer 指出的 naming overfit 痛点、以及 BFCL v4 format sensitivity 的评测趋势一致，属于审稿人能认可的现实动机。[16]

不建议继续投入的部分

- **把 C（multi-candidate latent policy + reranking / diffusion-like extension）当主线**：这会被拉进 best-of-n / verifier / inference-time scaling 的拥挤赛道，并且很难服务你“latent decision 不是压缩码”的主证据链。[15]
- **重型 RL 叙事**：ToolRL、TRM 等已经把“reward/PRM/credit assignment”作为中心问题做得很系统；你如果把 RL 放到主卖点，会让你的论文与它们正面硬刚而处于劣势。[17]
- **把 B 做成第二条独立贡献**：除非你要投 benchmark track，否则对方法论文不划算；更好的定位是“为 A 的可证伪性服务的评测协议”。[18]

主方向选择与研究命题改写

如果只能押一个主方向，我建议押：**“在 schema 变换下不变的 decision representation + execution-feedback posterior distillation”**，并把论文写成一个非常干净的 thesis：**定义清楚的 invariance 目标 + 一个最小可行的 posterior-distill 学习机制 + 一组能证伪的跨 schema 实验**。这样你才能最大程度避免被看成“CoLA for function calling” 或 “token hindsight 换空间”。

推荐主方向的一句话定义

用执行反馈学习一个对 schema 变换近似不变的 tool-use 决策表征，并证明它在跨 schema / 大 tool pool 下比 token-level hindsight 更可迁移。（对应 SDPO/OpenClaw 的正面挑战）[19]

推荐论文标题方向

- Schema-Invariant Decision Distillation for Executable Function Calling
- Execution-Feedback Distillation in a Schema-Abstracted Decision Space
- From Tokens to Decisions: Posterior Distillation for Robust Tool Calling

如何把 thesis 写得更干净，避免“CoLA/OPD 换皮”误读

你需要把贡献点从“我们也用 latent”改写成“我们 **定义并优化** 一个工具调用中长期缺失的结构性目标”。

我建议把核心 novelty 强行钉在两个句子上：

1. **问题定义层面的 novelty**：把 function calling 写成一个带 presentation layer（schema）的结构：
2. 目标不是学会某套 schema 的工具 token，而是学会一个对 schema 变换保持不变的 decision 表示；schema 只是 grounding 的坐标系。
   这一步要明确你研究的变换集合：tool renaming、argument renaming、doc paraphrase、以及你愿意承担的少数 argument reshaping 类型。并把它们当作“等价类”的生成器，而不是随手写的 stress test。 [4]
3. **学习机制层面的 novelty**：execution feedback 的使用方式不是 “repair output”，而是 “distill a better decision posterior”。
   你必须强调与 SDPO 的区别：SDPO 是 feedback-informed token distribution 的蒸馏；你要证明的是——**如果把蒸馏对象换成 decision representation，并加上 invariance 约束，就能在 schema 变换下获得更稳定的迁移。** [8]

推荐方法的最小闭环

把你草案里的模块堆叠砍到“一个闭环”，我建议最小闭环长这样：

- **Decision encoder**：从对话状态 s 产生一个中间表征 d（你可以继续叫 z，但要强调它是 decision representation，不是工具 ID 压缩）。
- **Grounding head**：条件于 (d, schema set T) 生成可执行 tool call y（最好强约束输出格式，避免把论文写成“格式修复工具”）。
- **Feedback posterior head**：执行 y 得到 feedback f（包括 tool output / error type / user correction），产生 posterior decision 表征 $d^+ = q(d \mid s, f)$。
- **核心训练信号**：让 prior decision d（或其分布）向 posterior decision d^+ 对齐（distillation / KL / regression），并加入跨 schema 变换的一致性约束（同一语义任务在等价 schema 下 d 应一致）。

这里我建议你**删掉/延后**：
- 可选 RL calibration（除非 pilot 证明“无 RL 做不到关键结论”）——否则会把论文拖进 ToolRL/TRM 的赛道。 [17]
- Multi-candidate / rerank / diffusion-like（C）——除非误差分析非常明确，否则会稀释主线并被认为是 best-of-n 工程。 [15]

为了防止“latent=压缩码”，我还建议你在最小闭环里强行加入一个**反自证**设计：
- **Counterfactual decoding test**（作为训练/评测协议的一部分，而非额外模块）：同一个 decision 表征 d，在不同 schema 变换下的 grounding 结果应保持语义正确（后面实验部分展开）。

这样写的好处是：你方法上看起来更像“一个干净的 learning principle”，而不是“拼装系统”。

不建议做的东西

- 不要把工作写成“我们提出一个新的 function calling benchmark”；BFCL 系列已经是主流基准，且正在扩展为 agentic evaluation，你需要的是“用它们证明你的 thesis”，而不是“替代它们”。 [20]
- 不要把工具表示学习（ToolWeaver/Toolscaler 的 structured tokenization）当成你的主要贡献点；那是另一条赛道，且已有强结果与大规模工具库实证。 [21]

- 不要在主实验里把 open-world web browsing 作为主 setting；BFCL v4 web search 属于 agentic 能力评估，干扰因素太多，会掩盖你要证明的“schema invariance + decision posterior”主张。 22

实验设计以证伪为核心

你要求“不要把实验当附属品”，我同意——而且你这条线如果不被严厉证伪，它就会被审稿人用一句话否掉：“看起来像 SDPO/OPD + consistency regularization”。

我把实验反过来设计：先挑最容易打脸你的实验，再挑能证明你不是 recipe 的实验，最后才是扩展。

Go / No-Go pilot

目标：用最小成本回答三个问题：
(1) 你的提升是不是 token-level hindsight 已经能吃到的？ 7
(2) 你的“latent decision”在 schema 变换下是否真的更稳（不是正则硬做出来的）？ 16
(3) 你是否有资格把 BFCL 当主舞台（以及用哪一版/是否需要修复版）？ 23

数据或 benchmark 子集
- 选 **BFCL v3** 的小子集（优先 Python 可执行类，便于你做 schema wrapping/alias），控制在 100–300 条规模。BFCL v3 明确包含 multi-turn/multi-step 设置，同时其评测流程说明避免使用 ReAct 等强 prompt trick，有利于公平对比“学习机制”。 24
- 同时准备一份“可控 schema 扰动子集”：对每个工具/函数做同构扰动（tool renaming、argument renaming、doc paraphrase），再挑一小部分做你能保证语义等价的 argument reshaping（例如把两个参数包装到一个对象参数里，但 wrapper 内部可还原）。

baseline（必须有强对照，否则论文主张站不住）
- Vanilla SFT function-calling baseline（同训练预算）。
- Hammer-style “function masking / irrelevant-function sensitivity” baseline（不用复现 Hammer 全套，只实现其核心思想：降低对命名/局部 token 线索的依赖）。Hammer 的动机与结论都直接指向 naming overfit，因此它是你必须击败的同类对照。 10
- Token-level hindsight baseline：用 SDPO/OPD 风格的“feedback-informed 生成 → 蒸馏回 policy”的最小实现（别只写“类似 OpenClaw/SDPO”，要真的跑一个可复现版本）。 7
- CoLA-lite without posterior: 做一个有中间 latent head 的两层结构，但不做 feedback posterior distillation（用来证明“不是 latent 本身带来的”）。 2

stress tests（pilot 必须用来证伪，不是装饰）
按优先级：
1) tool renaming + argument renaming（最直接检验是否摆脱 naming convention 过拟合，Hammer 已把它当关键痛点）。 25
2) irrelevant tools injection + enlarged tool pool（检验 decision 表示是否真在做“选择”，而不是记忆）。BFCL 本身强调 relevance/abstain 与 multi-step agentic 能力，你的扰动应与其目标对齐。 26
3) limited argument reshaping（只做你能证明语义等价的那几类；否则会引入 task drift）。
4) execution noise（把工具返回加噪或引入可控 error type，用来测试“posterior 学到的是 decision 更新”还是“看 error message 就能修输出 token”）。

核心指标（pilot 不要贪多，但要能支撑结论）
- 可执行成功率 / executable accuracy（BFCL 强调 executability，且提供执行/AST 等评测视角）。 27
- **Schema perturbation gap**：原 schema vs 扰动 schema 的性能差（这是你论文的生命线）。
- **Counterfactual decoding score**：用同一条样本在 schema A 上得到 decision 表示 d，然后在 schema B 上

只换 grounding（不重算 d）看是否仍能正确调用（这直接检验 d 是否 schema-invariant，而不是压缩码）。
- 与 token-level hindsight 的差距：尤其在 renaming/reshaping 下的差距。

成功标准（满足其一不够，必须同时满足）
- 在 renaming/argument renaming 的设置中，你的方法相对 SFT、Hammer-style masking、token-hindsight 均有**稳定且显著**优势（不是 0.2–0.5 个点的漂移）。[28]
- counterfactual decoding 不是随机提升：即“同一个 decision 表示跨 schema 可复用”这件事要成立，否则你的核心叙事（decision vs grounding）就是假的。
- ablation 显示：去掉 posterior distillation 或去掉 invariance 约束会显著退化（证明不是 recipe）。

失败标准（任一触发就该暂停主线或彻底改写）
- token-level hindsight（SDPO/OPD-lite）在 renaming/扰动下与你持平或更好（意味着你只是把既有方法换空间）。[7]
- 你的优势只出现在原 BFCL setting、在 perturbation 下消失（说明不是 schema-invariant decision）。
- counterfactual decoding 基本不成立（说明所谓 decision 表示并不可迁移；latent=压缩码或噪声容器）。

主实验

pilot 过了，主实验要回答“这是不是一篇方法论文”，而不是“我们在 BFCL 上刷分”。

主 benchmark（建议组合，而不是只押一个）
- **BFCL v3 multi-turn**：它明确给出 multi-turn/multi-step 的评测流程与终止逻辑，并强调避免引入 ReAct 等提示技巧，适合你去证明“学习机制而非提示工程”。[24]
- **BFCL v4 format sensitivity**：它系统化 prompt/format 维度的扰动，能帮你证明“不是靠格式/提示凑出来的提升”，同时也能检验你的 grounding 是否稳健。[11]

这里需要提醒：**BFCL 的多轮部分近期被指出存在大量系统性错误与评估/排名偏差，并出现了 audit/repair 与 outcome-aware evaluation 的工作**。即便你不使用其修复版，你也需要解释你如何规避这些问题（例如只用经过审计的子集、或对关键条目做 sanity check）。[14]

主要 baselines（要覆盖“审稿人最想用来否定你”的那类）
- SFT baseline（必备）。
- Hammer-style masking / irrelevance sensitivity（必备，同动机对照）。[25]
- Token-level SDPO/OPD-lite（必备，同反馈学习对照）。[7]
- 至少一个 reward-model / RL 方向对照（轻量即可）：例如 TRM（tool-call PRM）或 ToolRL（reward design + GRPO）的“思想对照”版本，目的是让审稿人无法把你的收益解释为“你只是没用更好的 RL 信号”。[29]

主要 ablations（不是凑表，而是围绕“空间替换”质疑）
- 去掉 posterior distillation（只保留两层结构）。
- 用 token-level hindsight 取代 decision-level posterior distillation（同预算）。[7]
- 去掉 schema invariance/consistency loss（看鲁棒性是否消失）。
- 去掉 schema conditioning 或把 grounding 合并回单头生成（检验“decision/grounding 分离”是否必要）。
- （若你坚持加 RL）去掉 RL calibration（证明 RL 不是主来源）。[30]

关键图表（必须把“不是 recipe”画出来）
- **鲁棒性曲线**：x 轴是扰动强度/工具池大小（enlarged pool），y 轴是 executable success；对比 SFT、Hammer、token-hindsight、你的方法。
- **Schema perturbation gap 分解**：分别在 tool renaming、arg renaming、doc paraphrase、arg

reshaping 上的 gap（告诉审稿人你到底在解决哪种鲁棒性）。
- **Decision reuse / counterfactual decoding 图**：展示同一 decision 表示在不同 schema 下的可复用率；这张图是你“latent ≠ 压缩码”的第一张王牌。

最重要的表应该展示什么
- 表 1：主 benchmark（BFCL v3/v4）上的总体结果 + 关键子类（multi-turn、format sensitivity）。[31]
- 表 2：schema 扰动套件上的鲁棒性结果（尤其 renaming/reshaping/enlarged pool）。
- 表 3：与 token-hindsight 的 “apples-to-apples” 对照（同 rollout 预算、同 feedback 输入限制），并包含消融（posterior vs token）。[7]

外部验证

目标：增强说服力，但不让主线跑偏到“通用 agent framework”。

我建议两类外部验证：

1) **WildAGTEval 作为“真实 API 复杂度”压力测试，但不建议当主实验**
WildAGTEval 强调现实 API 复杂度（spec + execution），并提供可组合的复杂度场景与大量配置；它更像“真实世界噪声与约束”的系统测试。[32]
问题是：它引入的干扰因素太多（文档冗余、执行噪声、约束组合），会让审稿人难以把提升归因到你想要的机制（decision posterior + schema invariance）。因此它很适合作为 Level 3 外部验证：你只需要证明“在更真实复杂度下，趋势不崩”，而不是靠它拿主结果。[33]

2) **HammerBench / 其他对话式 function calling benchmark 作为“交互鲁棒性”补充**
HammerBench 明确模拟移动助手多轮对话、意图/参数漂移、代词引用等，并指出参数名错误是重要失败来源；这与“decision/grounding 分离”叙事高度相关。[34]
它能帮你回答：“你的 decision 表示是否更抗 multi-turn 的语言漂移与参数扰动？”但同样，不要让它变成主战场。

关于 BFCL 更适合作为评测还是训练源
- 作为论文主线，我建议 **BFCL 主要用于评测**：它是公开的主流基准（ICML 2025 论文、v4 官方 leaderboard），适合做可信对比；但用它做大量训练会引入污染/过拟合质疑。[35]
- 甚至作为评测，你也需要面对“多轮条目的系统性错误与评估一致性”问题；EigenData 的审计/修复工作至少说明：对 BFCL-v3 多轮部分做 sanity check、或引用修复版/ outcome-aware 评估，会显著提升你实验可信度。[36]

关于 schema perturbation / renaming / reshaping / prompt variation / execution noise / unseen tools / enlarged tool pools 的价值与优先级
- 最高优先级（直接支撑 thesis）：tool renaming、argument renaming、irrelevant tools injection、enlarged tool pools。[37]
- 中优先级（强化“posterior refinement”而非 token repair）：execution noise（分层控制噪声类型）。[38]
- 中优先级（排除 prompt/format 工程）：prompt variation / format sensitivity（用 BFCL v4 的现成协议即可）。[11]
- 低优先级（容易扩成另一个方向）：unseen tools（如果你把它做成“开放工具检索/大工具库语义表示”，会撞上 ToolWeaver/Toolscaler 赛道）。[21]
- 谨慎使用（最容易引入 task drift）：argument reshaping（只做你能保证语义等价且可执行的少数变换）。

关于 C 方向（multi-candidate / reranking / diffusion-like）现在该不该做
不该作为主线，也不该在主实验里强行加戏。原因是：

- reranking/verifier 增强已是成熟套路（例如 Toolken+ 明确做 top-k rerank + 校验）；你很难把它和“decision posterior 学习”绑成不可分割的贡献。[39]
- “cycle verifier / inference-time scaling”类工作也在增长，容易把审稿人注意力拉离你的核心命题。[40]

只有当你完成 A 主线后，通过误差分析发现失败主要来自“多策略候选之间的选择”而非“schema grounding/decision shift”，才值得把 C 当附录级扩展。

更新后的研究方案草稿

Problem statement

现有 function calling 的主流训练与改进路径，大多直接在 **tool-call token space** 上学习与修补：要么靠 SFT 数据覆盖，要么靠 rich feedback 做 token-level distillation（SDPO/OPD 风格），要么靠数据增强/掩码减轻命名与格式过拟合。[41]

但在真实部署中，schema/format 的“表层变化”（命名、参数键、文档风格、工具池大小、无关工具注入）会显著破坏泛化；这在 BFCL v4 的 format sensitivity 与 Hammer 的命名过拟合分析中已被系统观察到。[42]

Core hypothesis

可执行 function calling 的关键学习对象不是 schema 绑定的 tool-call tokens，而是一个对 schema 变换近似不变的 decision representation；执行反馈应主要用于蒸馏/更新该 decision posterior，从而在 schema 扰动与大工具池下获得更稳定迁移。[19]

Method sketch

最小闭环方法（不引入重型 RL，不引入 multi-candidate）：

- **Decision encoder**：将状态 s（对话历史 + 已有工具返回 + 当前工具集合描述）映射为 decision representation d。
- **Schema-conditioned grounding**：基于 (d, T) 生成可执行 tool call y（强约束格式，避免把贡献写成“格式修复器”）。BFCL 本身强调 AST/可执行评估维度，这里要对齐其评价方式。[26]
- **Execution-feedback posterior**：执行 y 得到反馈 f（tool output / error type / 用户纠正），输出 $d^+ = q(d|s, f)$。
- **Learning principle**：通过 distillation（例如让 prior 的 d 匹配 posterior 的 d^+）把“反馈后更合理的决策”压回无反馈的 decision encoder；并加入跨 schema 变换一致性（对同一语义任务，rename/paraphrase 下 d 应保持不变）。

对照组必须含 token-level hindsight（SDPO/OPD-lite）与 Hammer-style masking，确保你不是在复现既有收益来源。[19]

Why it is novel

这篇论文如果要成立，novelty 不能写成“我们用了 latent”。它必须被写成：

1) **一个明确的问题结构**：把 schema 当作 presentation layer，把“决策不变、grounding 等变”当作目标结构（而非后验解释）。这与 Hammer 指出的 naming overfit 现象形成闭环动机。[25]
2) **一个与现有 rich-feedback 学习路径不同的作用点**：SDPO/OpenClaw 把 feedback 作用在 token prediction；你把 feedback 作用在 schema-abstracted decision posterior，并用 counterfactual decoding 等实验显示其可迁移性。[7]
3) **一个为“latent 不是压缩码”量身打造的证据链**：不是靠自定义一致性指标，而是靠反事实解码、跨 schema 复用、以及与 token-hindsight 的同预算对照来证伪替代解释。[43]

Experimental plan

- **主实验**：BFCL v3 multi-turn + BFCL v4 format sensitivity（主表与关键消融），并配套你自建的 schema renaming/arg renaming/enlarged pool 套件。 [44]
- **关键对照**：SFT、Hammer-style masking、token-level hindsight（SDPO/OPD-lite）、CoLA-lite（无 posterior）。 [45]
- **外部验证**：WildAGTEval（小规模趋势验证，不做主结果）+ HammerBench（交互式参数漂移/代词等鲁棒性补充）。 [46]
- **评估可信度保障**：对 BFCL-v3 多轮条目做 sanity check，并明确说明你是否使用/借鉴审计修复与 outcome-aware 评估思想（至少要在附录回应该风险）。 [14]

Main risks

1) **decision representation 仍是压缩码**：正则化能让它“看起来稳定”，但不代表可复用决策。 [47]
2) **posterior teacher 作弊**：如果 $q(\cdot)$ 看到过多信息，收益可能来自额外监督而非学习对象改变（与 SDPO 类似的质疑路径）。 [12]
3) **鲁棒性收益被 Hammer-style masking / prompt variation 吞掉**：如果你打不过这些强工程基线，就不应继续押这条“新学习对象”叙事。 [16]
4) **评测基准噪声**：BFCL-v3 多轮条目的系统错误会污染结论（尤其当你追小幅提升时）。 [36]

Kill criteria

以下任一成立，就应当放弃“latent decision posterior refinement 是主线”的论文叙事（要么大改 thesis，要么转向别的主线）：

- **token-level hindsight（SDPO/OPD-lite）在 renaming/扰动 setting 下与您持平或更好**（同 rollout / 同反馈输入限制）。 [7]
- **counterfactual decoding 失败**：同一 decision 表示无法跨 schema 复用，说明 decision/grounding 分离是假命题。
- **你的收益主要来自 prompt/format 工程**：在 BFCL v4 format sensitivity 下优势消失或不稳定，或通过简单格式约束/修复即可复现收益。 [11]
- **去掉 invariance 约束后鲁棒性不变**：说明“schema-invariant decision”并非机制核心，你的论文主张不成立。
- **BFCL 评测噪声解释了主要差异**：在经过审计/修复或子集复核后，关键结论不复现。 [36]

（补充说明：你草案里“C 作为可选扩展”的定位是对的；但在上述 kill criteria 触发前，把 C 拉进来只会让论文更像工程堆叠，而不是方法论文。）

---

[1] [24] [31] [44] https://gorilla.cs.berkeley.edu/blogs/13_bfcl_v3_multi_turn.html
https://gorilla.cs.berkeley.edu/blogs/13_bfcl_v3_multi_turn.html

[2] [5] [13] [45] https://proceedings.mlr.press/v267/jia25e.html
https://proceedings.mlr.press/v267/jia25e.html

[3] [7] [8] [12] [19] [41] [43] https://arxiv.org/abs/2601.20802
https://arxiv.org/abs/2601.20802

[4] [10] [16] [25] [28] [37] https://openreview.net/forum?id=yVQcr4qjD6
https://openreview.net/forum?id=yVQcr4qjD6

[6] [21] https://arxiv.org/abs/2601.21947
https://arxiv.org/abs/2601.21947

9 https://arxiv.org/abs/2603.10165
https://arxiv.org/abs/2603.10165

11 42 https://gorilla.cs.berkeley.edu/blogs/17_bfcl_v4_prompt_variation.html
https://gorilla.cs.berkeley.edu/blogs/17_bfcl_v4_prompt_variation.html

14 23 36 https://arxiv.org/html/2603.05553v1
https://arxiv.org/html/2603.05553v1

15 39 https://aclanthology.org/2024.findings-emnlp.345.pdf
https://aclanthology.org/2024.findings-emnlp.345.pdf

17 30 https://arxiv.org/abs/2504.13958
https://arxiv.org/abs/2504.13958

18 20 26 35 https://openreview.net/forum?id=2GmDdhBdDk
https://openreview.net/forum?id=2GmDdhBdDk

22 https://gorilla.cs.berkeley.edu/blogs/15_bfcl_v4_web_search.html
https://gorilla.cs.berkeley.edu/blogs/15_bfcl_v4_web_search.html

27 https://gorilla.cs.berkeley.edu/blogs/8_berkeley_function_calling_leaderboard.html
https://gorilla.cs.berkeley.edu/blogs/8_berkeley_function_calling_leaderboard.html

29 https://openreview.net/forum?id=LnBEASInVr
https://openreview.net/forum?id=LnBEASInVr

32 33 38 46 https://arxiv.org/abs/2601.00268
https://arxiv.org/abs/2601.00268

34 https://arxiv.org/abs/2412.16516
https://arxiv.org/abs/2412.16516

40 https://aclanthology.org/2025.findings-acl.1266/
https://aclanthology.org/2025.findings-acl.1266/

47 https://openreview.net/pdf?id=DQMjemrVhe
https://openreview.net/pdf?id=DQMjemrVhe